

Technical Report

Development of an AI-Based Virtual Camera Tracking System for Coarse Alignment of Mobile Free Space Optical Communication (FSOC) Terminals. Smart India Hackathon SIH26169. Organisation: Department of Space / Indian Space Research Organisation, Space Applications Centre.

Sections follow the order the problem statement asks for: problem understanding, system architecture, software modules, tracking methods, AI methods, the application, test methodology, performance analysis, future improvements, with appendices for the scenario pack, metric definitions, the command line and the row-by-row parameter mapping. Figures referenced as `results/...` are produced by the software itself; the numbers in section 8 come from the regression batch (`fsoc-tracker batch`).

1. Problem understanding

1.1 Context

FSOC links carry data on a highly directional laser beam. Compared with radio they offer gigabit-to-terabit data rates, need no spectrum licence and are immune to electromagnetic interference, but a small angular pointing error breaks the link. Between moving platforms (satellites, UAVs) the two terminals must therefore run a Pointing, Acquisition and Tracking (PAT) loop. PAT has a coarse stage, which brings the remote terminal's beacon into the camera field of view and holds it there, and a fine stage, which locks the beam precisely. This work addresses the coarse stage.

1.2 The engineering problem in the problem statement

Developing coarse alignment algorithms on hardware needs a camera, a pan-tilt mechanism and optics. The problem statement asks for a software platform that replaces them: a configurable virtual scene with one or more moving beacons, a movable virtual camera with realistic rate limits, a configurable set of disturbances, an automatic detector and tracker that steers the camera, live statistics, and an automatically generated performance log.

1.3 Reading of the specification

- The scene ("screen", $\geq 2000 \times 2000$ px) is what the tracker observes each frame: the problem statement's "simulated video stream". The 640×480 px camera window (4×3 deg) is the controlled output and represents where the terminal is pointing. Its centre starts at the screen centre and moves at 5 to 10 deg/s at most.
- With one screen pixel equal to one camera pixel, the instantaneous field of view (IFOV) is $4 \text{ deg} / 640 \text{ px} = 0.00625 \text{ deg} = 22.5 \text{ arcsec}$ per pixel; the 10 px tracking specification is 0.0625 deg; at 5 deg/s and 30 Hz the window moves at most 26.7 px per frame; the maximum platform motion of 20 px per frame therefore consumes 75% of the slew budget.
- Benchmark 2 replaces the simulated scene with the evaluators' .mp4. The tracker and camera control run unchanged on the video frames; the output is the per-frame centroid and the timing and lock metrics. Given the evaluators' positions as a truth CSV (frame or t, x, y; short gaps in a file sampled every few frames are filled in for lock only), tracking error, centroiding error, RMSE and a true lock retention are computed against them; without it lock is judged from the tracker's own estimate, which can overstate it.
- Two error terms appear in the specification and are both logged: tracking error (true beacon to window centre) and centroiding error (measured centroid to true centroid).

1.4 Points the problem statement leaves open, and the choice made

Question	Choice	Reason
Does the tracker see the whole screen or only the camera window?	Whole screen by default; a hard mode restricts it to the window	"Observe the surrounding environment" and "simulated video stream"; a blind sweep at 5 deg/s cannot meet 2 s acquisition for far corners
Screen pixels to degrees	One screen pixel equals one camera pixel: 12.5 deg across 2000 px	The only anchor given is 4 deg over 640 px
Meaning of "tracking error" and "centroiding error"	Tracking: true beacon to window centre. Centroiding: measured to true centroid. Both logged with printed definitions	The PS uses both terms without defining them
Meaning of "lock"	TRACK state with the estimate within 30 px of the window centre	Acquisition needs a capture criterion; 20 px was tried and only lowered retention under vibration without lowering error
A sustained 20 px per frame platform shift	A bounded sway with that peak speed, amplitude at most 20% of the screen	A sustained shift leaves the screen in seconds
How the "designated moving target" is designated (row 8)	By appearance (default), by a start cue (as an operator or GPS/ephemeris cue would give) or by a point (a click or typed x,y)	The PS does not say; identical look-alikes cannot be told apart by appearance alone
Role of AI	Classical detector first; CNN fills gaps and targets faint beacons; trained on the simulator's exact labels	Keeps frame rate and reliability independent of the model

1.5 Numbers that shape the design

Quantity	Value	Derivation
Angular size of one pixel	22.5 arcsec	4 deg / 640 px
Allowed tracking error	0.0625 deg	10 px x 22.5 arcsec
Maximum window motion	26.7 px per frame	5 deg/s / 30 Hz / 0.00625 deg per px
Platform motion at the PS maximum	20 px per frame, 75% of the slew budget	row 25 against row 13
Window-sized areas per screen	about 13	(2000/640) x (2000/480)
Centre to corner with the target known	0.95 s	max(4.25 deg, 4.75 deg) / 5 deg/s, both axes moving together

2. System architecture

DISPLAY (PyQt6)	scene view, camera view with HUD, live plots, parameter panel
INSTRUMENTATION	telemetry bus -> <label>_frames.csv, _summary.json, _report.pdf
SIMULATION LOOP	fixed step at the camera update rate; deterministic from (scenario, seed)
WORLD	scene, beacons, camera window and gimbal, disturbance chain, renderer
PERCEPTION	FrameSource (simulator video), classical detector, CNN detector, sub-pixel centroid, association and identity, IMM estimator
CONTROL	state machine, feedforward + PI rate controller, gimbal limits

One `Simulation` object drives every frame for the GUI, the command line and the tests, so all three execute the same code path. A `FrameSource` abstraction yields frames from the simulator or from a video file; nothing downstream knows which.

Data flow per frame: kinematics advance; the gimbal integrates the previous command; the renderer draws the scene and applies the disturbance chain; the tracker detects, measures, associates and updates the estimator; the state machine chooses what to aim at; the controller emits a rate command clipped by the gimbal model; a telemetry record is written.

3. Software modules

Module	Purpose
engine/config.py	Typed configuration; one field per problem statement parameter row; YAML load and save.
world/scene.py	Backgrounds: starfield, terrain, gradient, flat.
world/targets.py	Beacon kinematics: line, circular, figure of 8, random (Omstein-Uhlenbeck), spiral, sinusoidal, user-defined waypoints, static.
world/camera.py	Gimbal: pose in degrees, rate and acceleration limits, command latency, IFOV conversions.
world/disturbance.py	Extinction, turbulence (wander, scintillation), PSF blur, platform sway, vibration, camera exposure gain, frame loss, Poisson (a Gaussian approximation), Gaussian and salt-and-pepper noise, in physical order; read and shot noise use independent random planes.
world/renderer.py	Draws the scene with sub-pixel beacon placement; returns ground truth.
world/sprites.py	Beacon shapes (square, circle, gaussian, cross, ring, diamond, custom mask) at width x height, shared by the renderer and the detector's width calibration.
perception/detect.py	Classical detector (median, background subtraction, matched filter, adaptive threshold, connected components, shape filters), sub-pixel centroid (centre of gravity plus 2D Gaussian fit), CNN heat-map detector (ONNX).
perception/estimator.py	IMM over constant velocity, constant acceleration and coordinated turn.
perception/egomotion.py	Frame-to-frame picture shift by phase correlation.
control/tracker.py	State machine SEARCH, VERIFY, TRACK, COAST, REACQUIRE; gating; appearance signature for identity; adaptive measurement noise.
control/controller.py	Feedforward plus PID rate controller (kd 0 by default) with latency lead, deadband and anti-windup.
engine/simulation.py	The run loop and telemetry record.
engine/metrics.py	Metric definitions and specification pass/fail.
engine/checks.py	Scenario check: clamped inputs, values beyond the PS, near-limit settings, physical limits.
engine/report.py	PDF report.
gui/app.py	Desktop application.
cli.py	run, video, batch, gui commands.

4. Tracking methods

4.1 Detection and centroiding

A 3x3 median (5x5 added automatically under heavy salt and pepper) removes specks. A 31x31 box blur estimates the smooth background, which is subtracted in signed 16-bit arithmetic (an unsigned subtraction clips the negative half and quantises the filtered response, which buried faint beacons). A Gaussian matched filter at one third of the configured beacon size raises the SNR of an extended spot against point noise and stars. The noise level is the median absolute deviation of the filtered image, so stars and the beacon itself cannot inflate the threshold they are measured against. Pixels above level + k sigma ($k = 4$) are labelled by connected components; blobs are kept if their area, aspect ratio and fill are consistent with a spot. Each blob is scored by matched-filter SNR, peak brightness and agreement with the expected size of the designated beacon. The leading candidates are refined by an intensity-weighted centre of gravity on a background-subtracted patch followed by a 2D Gaussian least-squares fit, giving centroids to about 0.01 px on clean frames and 0.1 to 0.2 px under heavy noise. Among several acceptable candidates the designated beacon is chosen by the fitted width expected from its configured size and shape (rows 9 and 10), a measure that does not move with the noise level, then by brightness.

Faint beacons (track-before-detect). After extinction a dim beacon can sit at three to six sigma per frame, where a single-frame threshold either misses it or floods the tracker with noise. When no candidate reaches the acquisition confidence, the search switches to a track-before-detect path: the detector runs at $k = 3$ with a narrower matched filter on a moving-target residual (the frame minus a running mean of the picture, which holds the stars and the sky and not a moving beacon), and the resulting weak candidates are linked frame to frame into chains with a consistent velocity. A chain hit in at least six of the last eight frames, with a mean matched-filter SNR above 3.5 and a fitted width near the designated beacon's, is promoted to a provisional track that seeds the estimator with the chain's velocity and must survive a stricter verification (five of six frames). While following a faint target the association gate is small, the strongest response inside it wins, and a running quality measure drops the track back to the chain search when what it accepts is no better than noise. On the low-light faint scenario this raised acquisition from none to 1.5 to 2 s with 93 to 96% lock retention on four of five seeds. The chain linking was a Python loop over up to 400 candidates x 600 chains; it is now vectorised with NumPy, in the same greedy order and with identical results. On `lowlight_faint` (30 s) the 99th-percentile frame time fell from 149 to 160 ms to 26 to 33 ms and the true rate rose from 68 to 81 FPS to 95 to 96 FPS. Two guards keep a faint track on the beacon. At 3 to 6 sigma the sub-pixel refit could slide onto a neighbouring noise clump: the centre jumped several px and the width ballooned to 7.5 px, and one such measurement kicked the velocity off the beacon. In faint mode the raw detection is therefore kept when the refit moves more than 3 px or its width exceeds 2.5 times the track's recent median width, and a candidate much wider than that is not associated. The CNN is not used while a faint track is active (section 5.1). On `lowlight_faint` seed 7 the run went from 86 px error and 78.6% lock to 2.8 px and 94.8%; over seeds 0 to 9 acquisition is 0.80 to 2.00 s, error 2.4 to 3.5 px and lock 94.0 to 97.8%.

4.2 Estimation

An IMM runs three Kalman filters (constant velocity, constant acceleration, coordinated turn) and blends them by likelihood. Process noise is sized for the accelerations the problem statement produces (beacon paths to $\sim 100 \text{ px/s}^2$, platform sway to $\sim 600 \text{ px/s}^2$). Measurement noise adapts: the zero-mean part of the innovation sequence is treated as vibration and added to the measurement variance, so the estimator smooths jitter instead of chasing it, while a consistent innovation (lag) is not mistaken for noise.

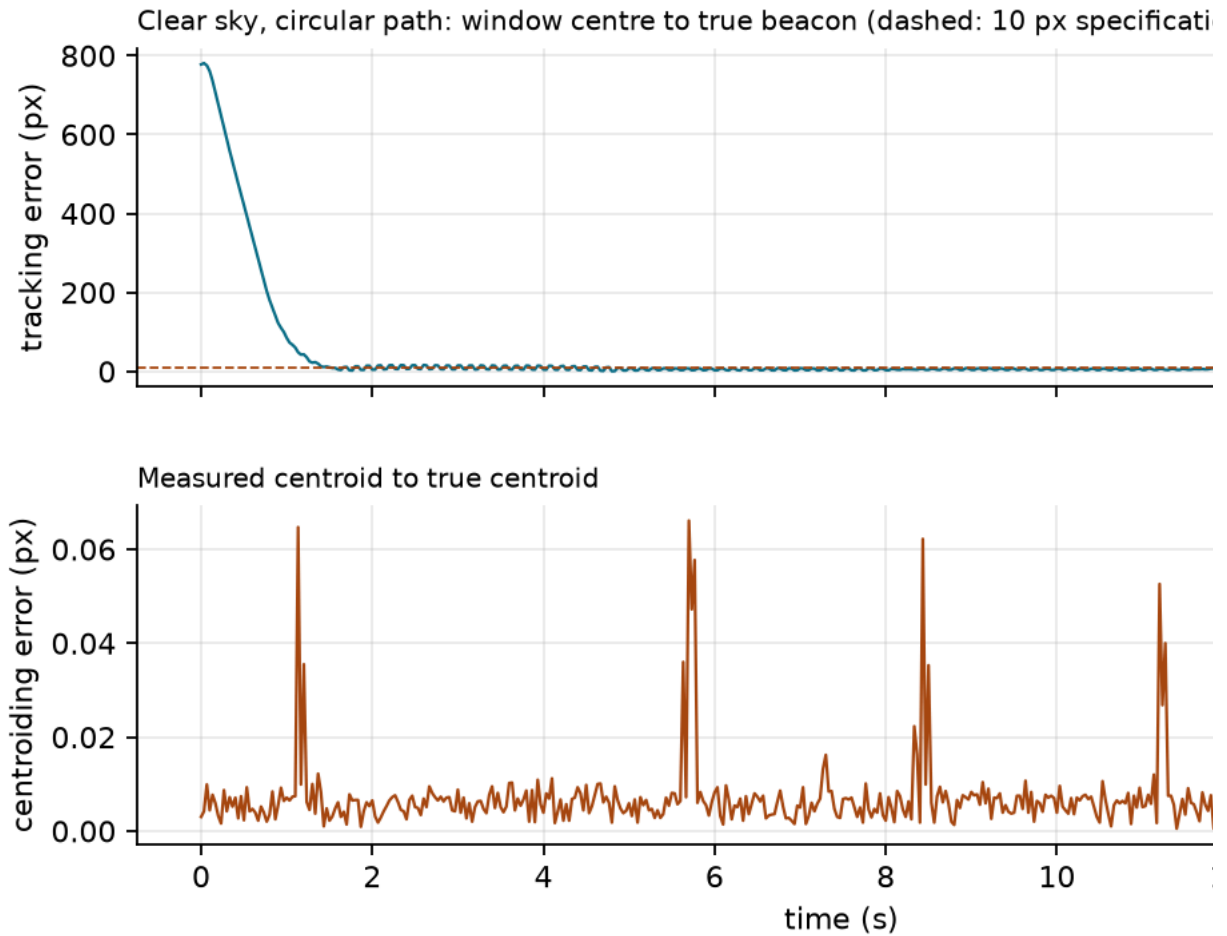
4.3 State machine and identity

SEARCH detects on the whole observed picture. VERIFY requires 3 of 4 consistent frames before declaring lock, so a noise speck is never counted as acquisition. TRACK searches a region around the prediction, gates candidates by Mahalanobis distance and rejects those whose appearance signature (area, peak, PSF width) differs strongly from the beacon being followed, which holds identity through crossings with decoys. COAST propagates the prediction through short dropouts; REACQUIRE widens the search with the growing uncertainty and falls back to SEARCH if it exceeds the screen. A coasting guard sends the tracker back to a whole-scene search at once when, while coasting or re-acquiring, the estimate lies more than the search window (160 px) outside the picture. Without it the estimate could run away: on `lowlight_faint` seed 2 (30 s) it drifted far off the screen, with centroid error up to 1268 px, tracking error 158 px, lock 75.1% and a 4.53 s re-acquisition; with it, 5.9 px, 96.2% and 0.07 s. In hard mode (tracker restricted to the window) SEARCH drives an outward spiral. While searching, candidates are re-measured on the current frame (before 2026-09-23 a stale or missing picture was used after a loss; fixed with the regression batch unchanged).

Designation (row 8, "a designated moving target"). Every target has a name; all are detected, `targets[designated]` is followed and scored. Mode `appearance` picks it by configured shape, size and brightness; `start` also gives its start position; `cue` gives a point near it (a click on the preview or on a video's first frame, or typed). With a cue the search takes the strong candidate nearest the cue, and after a loss the last estimate becomes the cue. In appearance mode, search frames where another spot scored within 0.15 of the chosen one are counted as ambiguous frames and reported. Tracking every beacon at once was not built: the PS metrics are for one target and one camera.

Scenario check. Every numeric input is clamped to its accepted range in the engine, so no front end can pass an impossible value (a web test had taken salt and pepper 13, meant as 13%, as a fraction and blacked out the frame). `engine/checks.py` then notes values beyond the PS, with the row named, near-limit settings (the designated beacon above 70% of the camera turn rate: it can be done, with little margin), and physical limits: a beacon faster than the camera turns (800 px/s at 5 deg/s), jitter plus platform above 26.7 px/frame, look-alikes in appearance mode. The notes appear in the panel, at Start, in the end dialog, on page 1 of the report and in the summary.

Figure 1 shows both error terms on a clear circular path: the window settles within about a second and holds 6 to 8 px while the centroid stays within 0.01 px of the truth.



4.4 Control

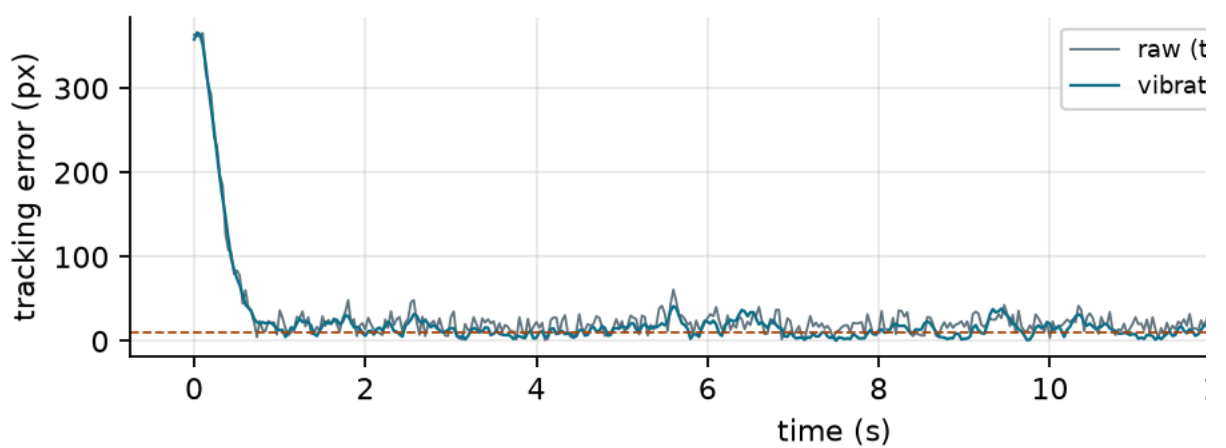
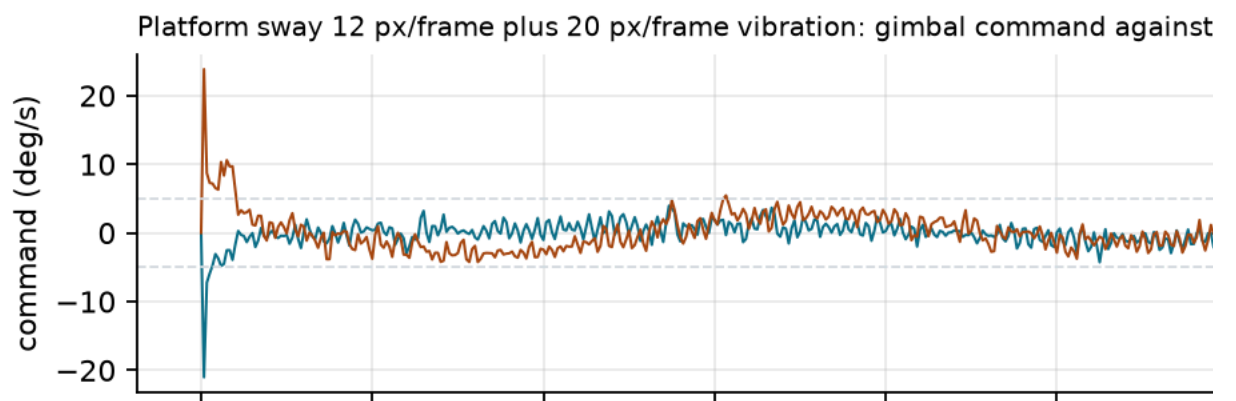
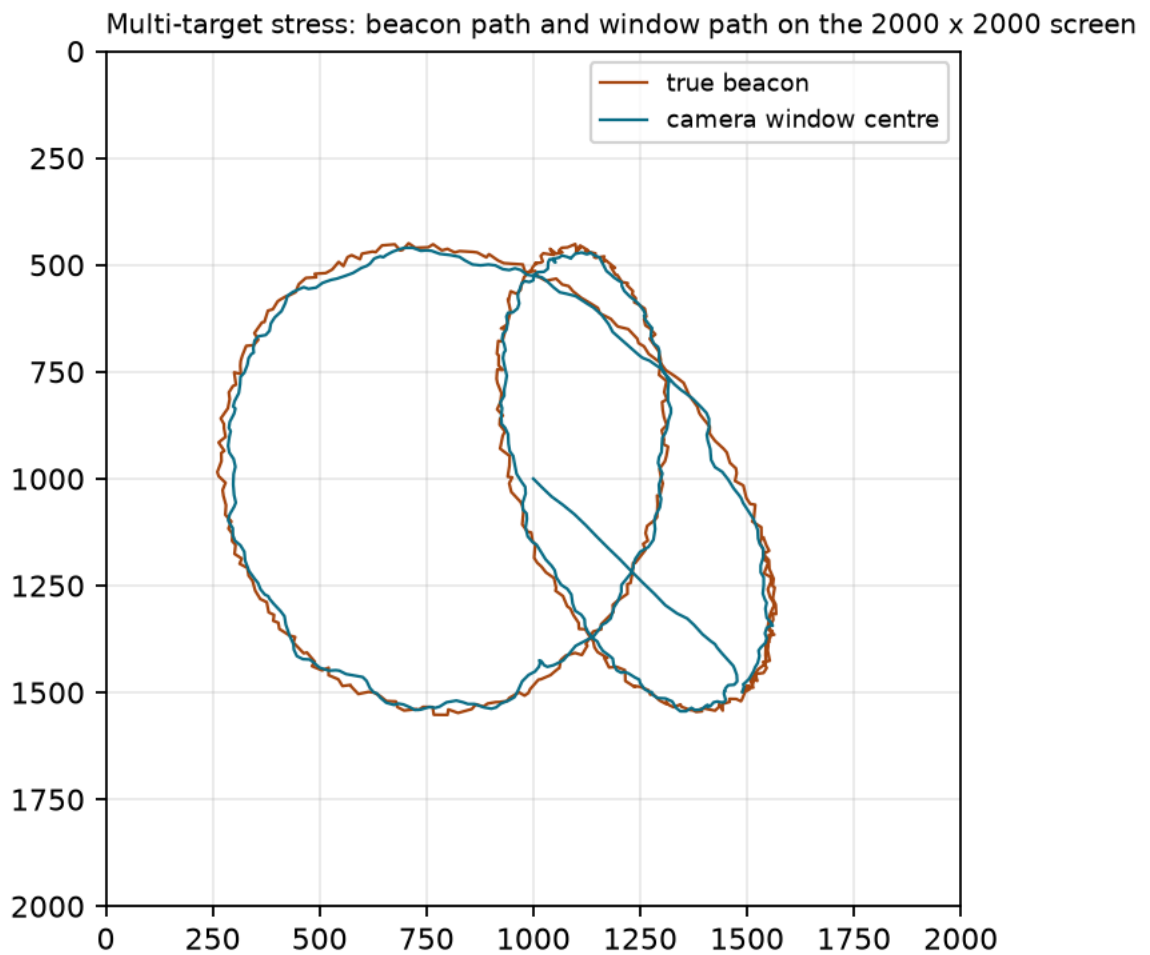
The angular error is the IFOV times the pixel offset from the window centre to the target position led by the command latency (velocity and acceleration), with the window's own motion led likewise. The command is feedforward of the target angular rate plus PI on the error (kp 5, ki 0.8), with a 1.5 px deadband and integrator clamping while saturated. The derivative gain is 0. At 0.3 it acted on an error that reaches the controller one frame late and in whole pixels, and drove a limit cycle of about ± 10 px: on a still beacon the camera oscillated forever (mean 6.4 px, peak 15.6 px). With kd 0 a still beacon is held within 4 px (mean 1.6 px), and tracking errors over the pack dropped by about two thirds. The PS audit's Shall 6 check found it (section 7). The gimbal model applies rate saturation (5 to 10 deg/s), acceleration saturation and a one-frame latency, and reports saturation to the log.

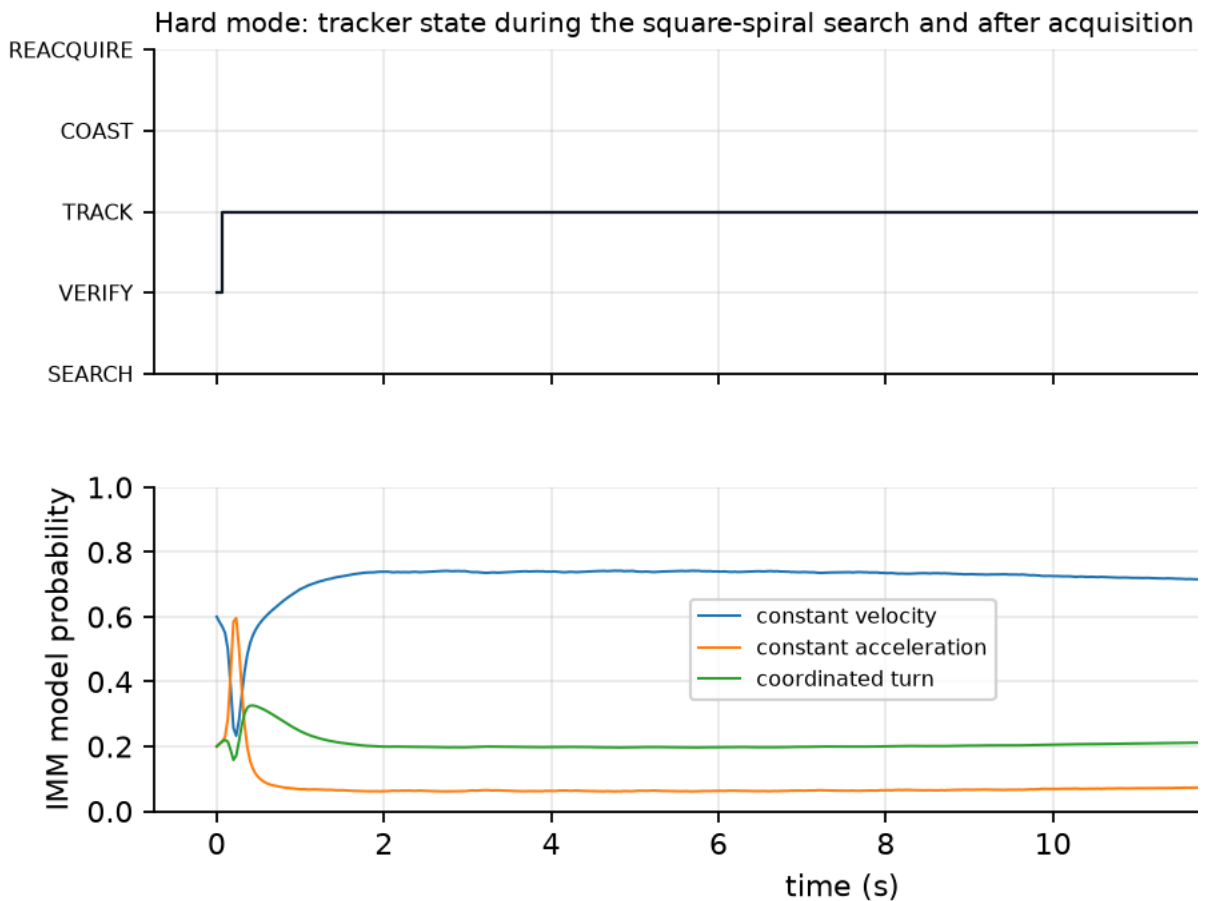
Turn-limited lead. The acceleration term advances the estimated velocity by a fixed 0.25 s, a straight-line extrapolation. On a tight curve this points the feed-forward off the path. The estimator itself was fine: its position is within 0.4 to 0.7 px of the truth at every speed, and its velocity lags by 1 to 2 frames. The lead is therefore capped so the path turns by at most 0.1 rad over it (`Controller.TURN_MAX`); slow or gently curving targets keep the full lead. Measured on a 450 px circle (12 s, seed 0, with kd 0.3), mean tracking error:

Angular speed	Before	After
1 deg/s	6.3 px	7.1 px
2 deg/s	6.3 px	8.4 px
3 deg/s	16.3 px (fail)	8.9 px
4 deg/s	34.3 px, 14 to 19% lock	8.4 px, 100% lock

The new `fast_circular` scenario (4 deg/s, 640 px/s, 80% of the 5 deg/s turn rate; 15 s, seeds 0 to 2) now gives acquisition 0.67 to 0.70 s, 4.6 to 5.3 px and 100% lock.

Figure 2 shows the beacon and window paths in the multi-target stress scenario (haze, salt and pepper, Gaussian and Poisson noise, circular platform sway, vibration, two decoys): the window path follows the figure of eight of the designated beacon and ignores the decoys. Figure 3 shows the gimbal command against its limit under platform sway plus vibration, and the raw and vibration-removed tracking errors. Figure 4 shows the state machine and the IMM model probabilities during a hard-mode run: a square-spiral search, verification, then track.





5. AI methods

5.1 Role

The classical detector is fast and accurate in clear conditions but fails in fog, rain, low light and heavy noise, where "the brightest compact blob" is the wrong answer. A small fully convolutional network then provides the measurement. It runs only when the classical confidence is below a floor, and only on a 128 x 128 region around the prediction, so speed and reliability never depend on the model. It is a gap filler: in the standard scenarios it supplies 0% of the measurements. It is not used while a faint track is active: it was trained on visible beacons, and on a 3 to 6 sigma patch its peak was often noise. The model file is looked up in the working folder and in the package folder, so an installed command started from another folder still loads it.

5.2 Network and training

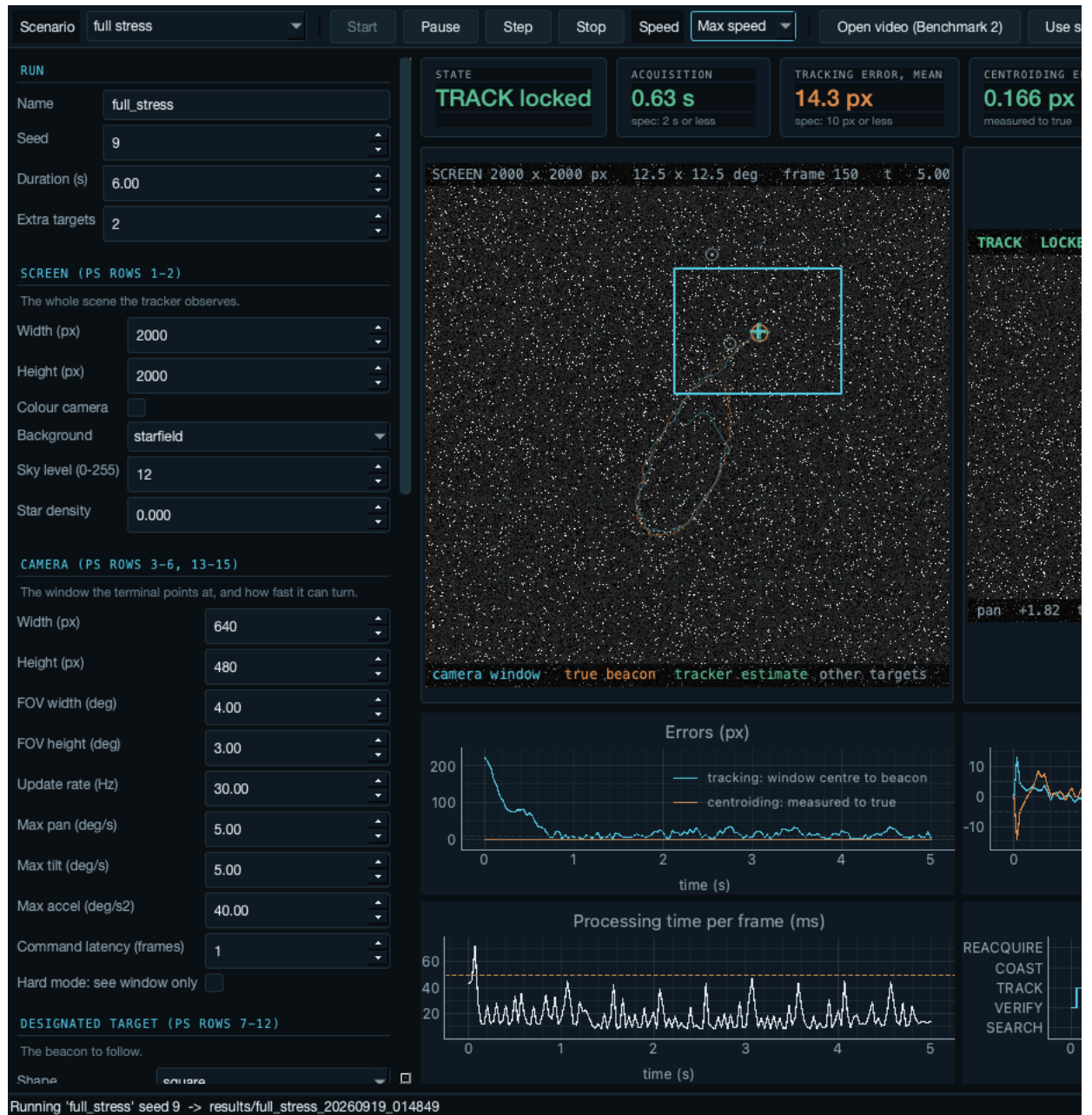
Three-level U-Net-style encoder with skip connection, about 0.1 M parameters, input a normalised 128 x 128 patch, output a 64 x 64 heat map with a Gaussian bump on the beacon. Loss: weighted binary cross-entropy. Training data are rendered by the simulator itself under randomised backgrounds, beacon shapes and sizes, decoys, atmosphere presets and noise; the simulator knows the beacon position, so labels are exact and unlimited. The model is exported to ONNX and run with ONNX Runtime on CPU in a few milliseconds. Sub-pixel position comes from a soft-argmax on the heat map followed by the same Gaussian refinement used by the classical path.

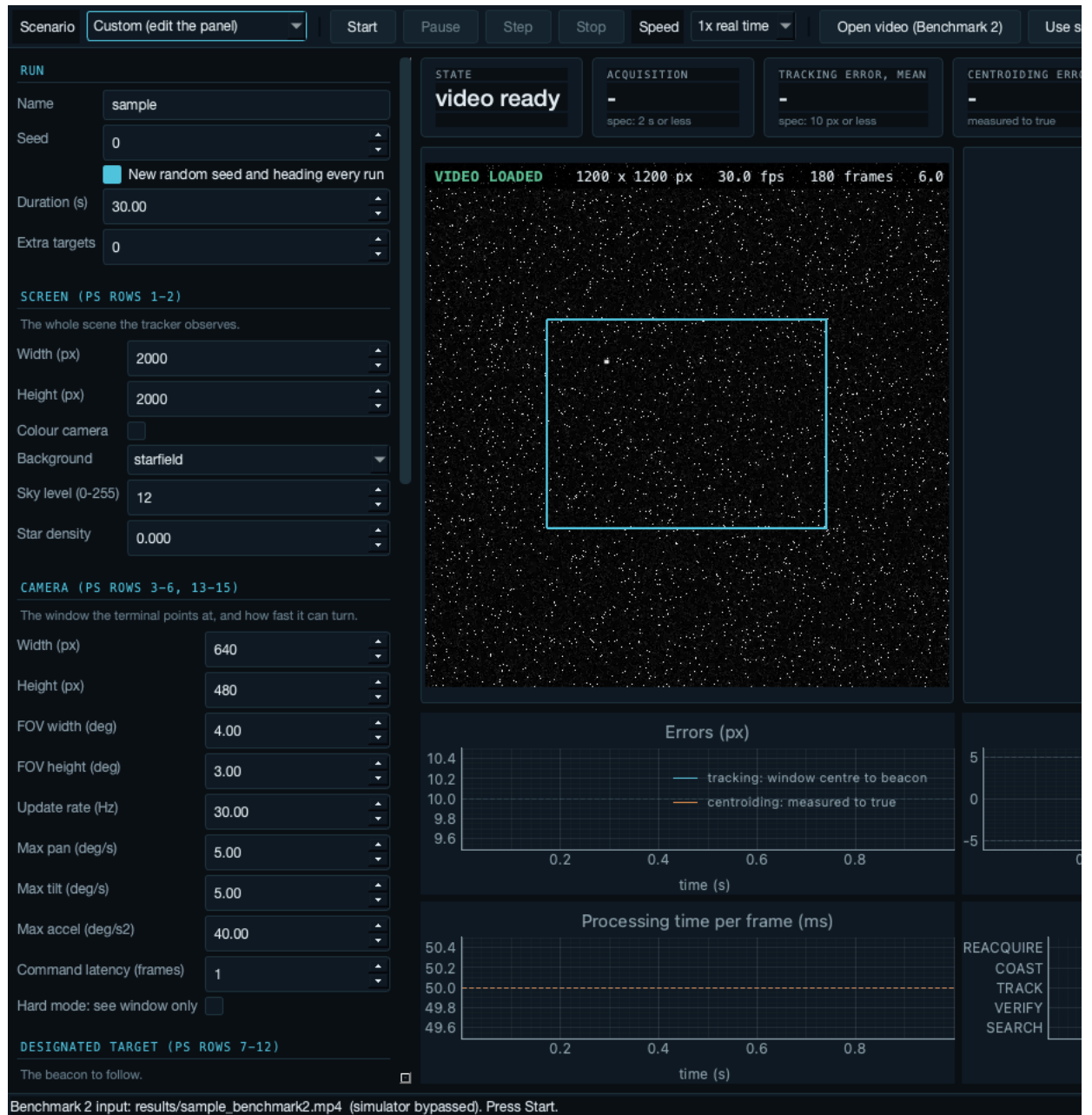
6. The desktop application

The deliverable is a standalone desktop application (PyQt6, packaged with PyInstaller). Its window has a parameter panel with one control per problem statement row and a tooltip naming that row; six live tiles that turn green when the specification is met (state, acquisition time, mean tracking error, mean centroiding error, lock retention, FPS); a scene view of the whole screen with the camera window, trails, a 2 degree grid and a legend; a camera view of what the window sees with the capture ring, the pointing-error vector, the detection box, the prediction with its uncertainty ring and a 1 degree scale bar; four live plots; and a telemetry column with every internal quantity. The Run section chooses the designated target by name, the a Designated tick box (the mode is automatic: appearance, plus the start position when look-alikes exist), an identical look for extra targets, and which target the Target section edits; below it the scenario check is shown live. Before a run the scene view previews $t = 0$ with every target named, and a click makes a target the designated one. A scenario picker, playback speed control and keyboard shortcuts support the ten to fifteen minute functional demonstration. "Open video" bypasses the simulator for

Benchmark 2 and previews the file's first frame and facts before the run; the target's appearance stays editable as the description of what to look for, and a click on the first frame marks the beacon to follow. "Truth CSV" loads a ground-truth file for the video (a `<video>_truth.csv` beside it is picked up automatically); the report's source line names it. On a noisy_line clip rendered to .mp4 with its truth: tracking error 4.85 px, centroiding error 0.188 px; without the file, n/a. Every run ends with a dialog summarising the specification check and opens the PDF report on request.

Disturbances can be changed while a run is going, as the "shall" item on introducing disturbances into the camera feed describes: the Disturbances section stays live (the other sections are fixed for the run), a change reaches the next frame, the tiles restart from it and the plots and report mark it. A scenario can script the same changes with a `schedule`, and a change made live is saved in the run's scenario at the frame it took effect on, so the run replays exactly. Platform sway switched on, changed or off mid-run never makes the picture jump: it settles into the new pattern within 1.25 times the larger peak speed.





7. Test methodology

- Unit tests (tests/): IFOV and pose conversions, gimbal saturation and pose limits, determinism and screen bounds of every motion type, centroid accuracy on clean frames, sub-pixel refinement on a synthetic Gaussian, IMM prediction on a circle, closed-loop specification checks on a clear scenario and on a platform-plus-vibration scenario, a Benchmark 2 path that writes a synthetic video and runs the tracker on it, and target tests (width and height, every shape, clamping and PS-envelope notes, designation), and the review fixes (FPS as frames over processing time, a 4 deg/s target kept centred, a coasting estimate that cannot run off the screen, video ground truth giving errors, the CNN model found from any folder), and the PS audit fixes (a still beacon centred without oscillation, platform sway never above the set speed, a faint track not walked off by noise). 45 tests.
- PS audit (tools/ps_audit.py): measures every PS item by running the code: rows 1 to 25, the eight "shall" functions, the five deliverables, Benchmark 1, and Benchmark 2 with a rendered noisy video and its truth CSV. It writes docs/PS_AUDIT.md and exits with 1 if any check fails. The build runs it on the Linux job. 39 of 39 pass.
- Regression rule: any change to perception, estimation or control is run on the whole pack (15 s, seeds 0 to 2) before and after and compared run by run; no run may be worse.
- Scenario pack (configs/scenarios/): the four mandatory motions in clear conditions, heavy noise, fog, low light, platform sway with vibration, a multi-target stress case, identical decoys, mixed beacon shapes, a fast circle and hard mode: 16 scenarios plus an evaluator template.
- Camera effects beyond the PS table: clear line, 15 s: 10% frame loss gives 88.0% lock (every lost frame breaks lock) but 100% on the frames received, tracking error 2.59 px (2.35 without) and re-acquisition within 0.10 s; 25% loss: 71.0% lock, 3.04 px, 0.17 s. Exposure x4 clips the beacon at white and raises the centroiding error from 0.006 to 0.093 px; x0.5 changes nothing measurable.

- Handoff to fine pointing: measured on 15 s runs, clear line 2.20 s (lock at 1.07 s), full PS noise 1.70 s, fog 2.43 s, all held 100%; faint beacon 3.37 s, held 37.8%; platform sway plus shake, the PS maximum and full stress never reach it (the estimate does not stay within 10 px).
- Reproducible figures: `fsoc-tracker verify <run folder>` rebuilds every metric of a run from its per-frame CSV alone and checks its summary JSON, so a reader can confirm the reported numbers.
- Batch envelope: `fsoc-tracker batch --scenario configs/scenarios/*.yaml --seeds 0-N` runs every scenario over N seeds and writes `envelope.md` with mean and worst values.
- Every metric has a printed definition (section 8 of the user manual) so the numbers can be compared with the evaluators' own.

8. Performance analysis

All figures below are measured by the software itself on the 2026-09-23 regression batch (`fsoc-tracker batch` , 15 s per run, seeds 0 to 2; the faint beacon over seeds 0 to 9; laptop CPU). Ranges run over the seeds and, where a row groups scenarios, over those scenarios. Tracking error is the mean distance from the true beacon to the window centre over frames after acquisition; centroiding error is the mean distance from the measured centroid to the true centroid; lock retention is the share of frames in TRACK with the estimate within 30 px of the window centre. All values are from the 2026-09-23 batch (15 s, seeds 0 to 2; the faint beacon over seeds 0 to 9).

Scenario	Acquisition (s)	Tracking error (px)	Centroiding error (px)	Lock retention (%)	FPS	Spec rows 16 to 20
clear: line, circular, figure of 8	0.60 to 1.33	2.1 to 3.7	0.006 to 0.007	100	193 to 219	pass
clear, random walk	0.67 to 1.03	5.1 to 6.3	0.006 to 0.007	100	189 to 210	pass
noise: salt and pepper 10%, Gaussian sigma 20, Poisson	0.67 to 1.00	2.4 to 3.7	0.19	100	103 to 109	pass
fog and low light	0.63 to 1.57	2.5 to 3.7	0.08 to 0.15	100	117 to 124	pass
fast circle, 450 px at 4 deg/s	0.67 to 0.70	4.6 to 5.3	0.006	100	200 to 211	pass
identical decoys, designation start	0.60 to 0.63	3.3 to 3.5	0.025 to 0.027	100	97 to 99	pass
beacon shapes, 8 x 18 px rectangle among other shapes	0.67	2.6 to 3.1	0.04 to 0.06	100	88 to 97	pass
low light, faint beacon at 3 to 6 sigma per frame, 10 seeds	0.80 to 2.00	2.4 to 3.5	0.3 to 4.0	94.0 to 97.8	105 to 110	pass; seeds 7 and 8 at 94.8 and 94.0%, just above the 5% loss limit
multi-target stress: 3 beacons, haze, noise, sway, vibration	0.30 to 1.03	10.3 to 11.3	0.16 to 0.23	98.2 to 100	115 to 118	identity held; error above 10 px from the vibration
platform sway 12 px/frame + vibration 20 px/frame	0.70 to 0.80	19.0 to 19.4 (12.2 to 12.5 with the vibration removed)	0.02	99.3 to 100	76 to 86	tracking error above 10 px: the truth itself moves 20 px per frame
platform at the PS maximum, 20 + 20 px/frame, 5 and 10 deg/s	0.73 to 0.97	21.7 to 23.5 (16.2 to 18.4 with the vibration removed)	0.02 to 0.18	93.4 to 97.2	78 to 85	documented physical limit; same at 10 deg/s, so the random vibration is the limit, not the motor
hard mode, tracker sees only the window	2.83 to 11.97	2.6 to 3.7	0.007 to 0.029	100	113 to 139	acquisition beyond 2 s: a blind sweep at 5 deg/s (search, by design)

FPS over the whole pack is 69 to 216.

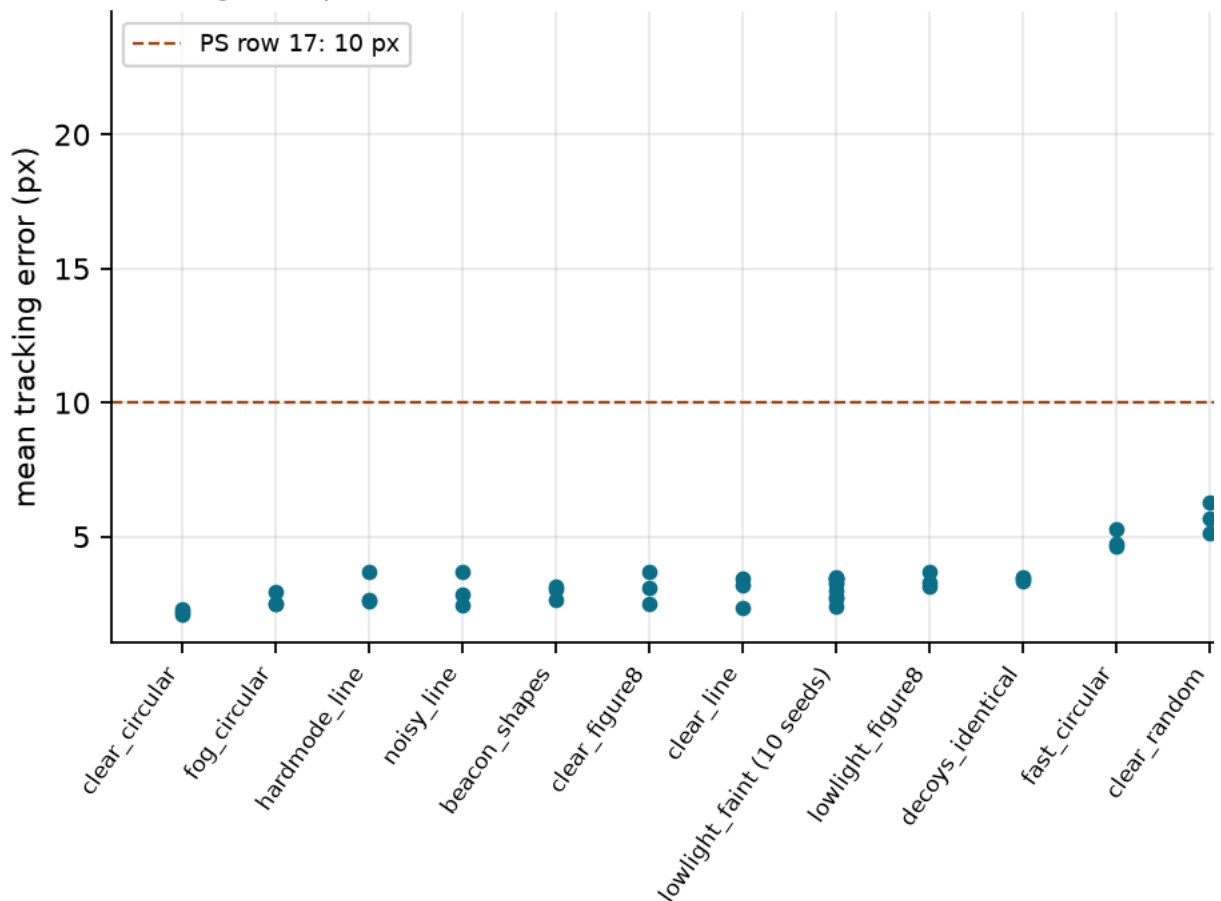
Discussion.

- Centroiding accuracy is set by the sub-pixel Gaussian fit and holds at 0.006 to 0.007 px in clear air and 0.19 px under the heaviest noise the problem statement lists. This is the quantity compared in both benchmark rounds.
- Tracking error on smooth paths is 2.1 to 3.7 px, inside the 10 px specification, with lock at 100%. The random walk, at 5.1 to 6.3 px, is no longer at the limit; it stays the highest clear case because its accelerations are unpredictable by construction: the controller can only lead what the estimator can extrapolate.
- Under vibration the raw tracking error exceeds 10 px because the true beacon position jumps up to 20 px every frame (19.0 to 19.4 px on `platform_jitter`); the vibration-removed figure is the part a rate-limited gimbal can physically follow.

Both are printed in every report.

- At the maximum platform motion (20 px per frame sway plus 20 px per frame vibration) lock is 93.4 to 97.2% (`platform_max` and `platform_max_10degs`, 15 s, seeds 0 to 2), error 21.7 to 23.5 px (76 to 88% on 2026-09-19, 88.5 to 94.8% after the review fixes). The binding limit was measured, not assumed: with the gimbal at the 10 deg/s the PS allows (`platform_max_10degs.yaml`) slew saturation falls from about a third of frames to 2%, yet lock and error do not improve. The vibration is the limit: a random 20 px jump of the picture every frame is unknowable before the frame arrives, so each frame the pointing is off by roughly the jump, the estimator must average many noisy frames, and the 30 px lock criterion flickers. No controller design removes this; a wider FOV or a sensor larger than the 640 x 480 window (electronic stabilisation) would, and neither is within the PS. Documented, not tuned away.
- Designation (15 s, seeds 0 to 2). `decoys_identical` (three identical look-alikes starting apart, designation start): 3.3 to 3.5 px, 100% lock. `beacon_shapes` (an 8 x 18 px rectangle among other shapes, appearance): 2.6 to 3.1 px, 100% lock. Before the designation modes, identical decoys starting apart with appearance only (20 s) gave the designated beacon in 1 of 5 runs and never acquired it in 3; with the start cue 5 of 5 pass (0.60 to 0.73 s, 5.7 to 6.6 px, 100%). Limit: look-alikes that start at the same point cannot be told apart at the start; even with the start cue those runs held 8 to 25% lock. After these changes all 42 earlier batch runs are identical.
- After the review fixes (15 s, seeds 0 to 2, against the committed merge): 38 runs the same, 10 better, 0 worse, 3 new. `full_stress` 11.4 to 12.1 px at 100% lock (was 13.7 to 14.4 px, 98.4 to 98.8%); `lowlight_faint` 96.4 to 97.8% lock (was 90.9 to 95.5%; seed 2 error 12.25 to 7.22 px); `platform_jitter` seed 0 lock 96.5 to 98.6%. A reported failure (a 5 px beacon in rain with 10% salt and pepper, never acquired, about 3 FPS) could not be reproduced with our settings: figure of 8, 30 s, seeds 0 to 2, acquisition 0.77 to 1.37 s, 6.0 to 6.5 px, 100% lock, 86 to 99 FPS.
- After the PS audit fixes (kd 0, the figure-8 sway scaled to its setting, the faint guards; 15 s, seeds 0 to 2, against the previous commit): 0 worse, 5 better by the tool's lock rule (platform maximum lock), every other run in its "same" band, although tracking errors dropped by about two thirds (the comparison tool flags error increases, not decreases).
- Identity among decoys holds in four seeds of five; the failing seed loses the beacon during a sway excursion and re-locks a similar decoy. The designation audit recovers some cases; a stronger appearance model is future work. Figure 5 summarises the envelope: mean and worst-seed tracking error per scenario.

Tracking error per scenario and seed (15 s runs, 2026-09-23)



- Processing runs at 69 to 216 FPS on a 2000 x 2000 scene on a laptop CPU (true rate over the 16-scenario pack), against the 20 FPS requirement. FPS is frames over processing time (1000 / mean ms). The earlier mean of per-frame rates overstates it when frame times vary (faint beacon seed 2, 30 s: 92.0 said, 67.8 true) and is kept as `fps_inst_mean` for comparison only. Hard mode is slowest because full-window detection runs every frame during the sweep.
- The AI detector reached 8 px validation localisation after retraining and is kept as a gap-filling fallback; it supplied about half the measurements on a real 60 fps phone video and 0% in the standard scenarios, where the classical detector

never loses the beacon.

Against a simple baseline. To show what each part of the design adds, a deliberately simple tracker was run through the same simulator, gimbal and metrics (`tools/compare_trackers.py` , every scenario, seeds 0 to 2, 15 s; `docs/BASELINE_COMPARISON.md`): it takes the brightest spot and steers toward it with proportional control. Measured: ARGUS meets every PS limit in 33 of 48 runs, the baseline in 0 of 48; the baseline's tracking error is 27 to 40 px on clear skies against 2 to 6 px, it holds 1.6% lock among identical decoys and never acquires the faint beacon; it acquires faster on some clear skies (0.37 s against 1.03 s on the circle) because it has no confirmation step. The matched filter, the confirmation, the IMM lead, the gating and the identity signature each remove a failure the baseline shows.

9. Future improvements

- Switching the designated target in the middle of a run (scored in segments), changing the scenario live during a run, and manual camera control. Not built; the PS does not ask for them.
- Reinforcement-learned gain scheduling on top of the classical controller, trained in the same simulator.
- Dual-sensor PAT: a low-resolution wide-field finder for acquisition feeding the narrow field tracking camera. Still open. By default the tracker observes the whole scene, the documented reading of the PS; hard mode covers the window-only reading.
- Learned denoiser for extreme scintillation.
- Note on frame rate: the estimator-lag compensation is defined at the 30 Hz reference and scales with the camera rate, because the filter delay is a number of frames. Found on a 60 fps phone video, where the unscaled value over-led the target; at 30 Hz nothing changes.
- Track-before-detect for a static faint beacon: the moving-target residual removes anything static, so a beacon that does not move must still be found by the single-frame path.
- Beacon modulation with lock-in detection for identity in dense clutter.
- Hardware in the loop: the `FrameSource` and gimbal interfaces already isolate the simulator, so a real camera and pan-tilt unit can be substituted.

Appendix A. Scenario pack

File	Purpose	Beacon	Disturbances
clear_line.yaml	mandatory path 1	square 10 px, straight line 150 px/s	none
clear_circular.yaml	mandatory path 2	circle radius 450 px, period 14 s	none
clear_figure8.yaml	mandatory path 3	figure of 8, radius 500 px	none
clear_random.yaml	mandatory path 4	random walk, 140 px/s	none
noisy_line.yaml	row 21 at its stated levels	line	salt and pepper 10%, Gaussian sigma 20, Poisson
fog_circular.yaml	row 24	circular	fog preset, Gaussian sigma 8
lowlight_figure8.yaml	row 24	figure of 8	low light preset, Gaussian sigma 12, Poisson
lowlight_faint.yaml	beyond the classical detector	gaussian 8 px, intensity 150	low light, noise
platform_jitter.yaml	rows 23 and 25	circular, centred	linear sway 12 px/frame, vibration 20 px/frame
platform_max.yaml	rows 23 and 25 at the PS maximum	circular, centred	linear sway 20 px/frame, vibration 20 px/frame
platform_max_10degs.yaml	the same with the gimbal at the allowed 10 deg/s	circular, centred	shows the motor is not the binding limit
full_stress.yaml	rows 8, 21, 23, 24, 25 together	figure of 8 plus two decoys	haze, salt and pepper 5%, Gaussian 12, Poisson, circular sway, vibration 10
fast_circular.yaml	rows 13, 14, 17: a fast target	circle radius 450 px at 4 deg/s (640 px/s, 80% of the turn rate)	none
hardmode_line.yaml	tracker restricted to the window	line	none
decoys_identical.yaml	row 8, designation start	Remote terminal plus three identical look-alikes starting apart, paths crossing	Gaussian 6, Poisson
beacon_shapes.yaml	rows 9 and 10, designation appearance	8 x 18 px rectangle among a 16 px cross, 18 px ring, 14 px diamond, 12 px custom	none

Appendix B. Metric definitions printed in every report

Metric	Definition
Simulation duration	Last frame time minus first.
FPS (<code>fps_mean</code>)	Frames over processing time (detection, estimation, control): <code>1000 / mean processing ms</code> . Row 20. <code>fps_inst_mean</code> , the mean of per-frame rates, is higher when frame times vary; for comparison only.
Acquisition time	First frame in TRACK with the tracked beacon within the capture radius of the window centre, from <code>t = 0</code> . Row 16.
Tracking error, mean and maximum	Distance from the true beacon to the window centre over frames after acquisition. Row 17.
Tracking error, vibration removed	The same with the per-frame vibration subtracted from the truth.
Centroiding error	Distance from the measured centroid to the true centroid over frames with a detection; mean, maximum, RMSE.
Tracked rate	Percentage of frames after acquisition in state TRACK, regardless of centring; separates tracker failure from gimbal limits.
Lock retention rate	Percentage of frames after acquisition in TRACK with the beacon within the capture radius. Target loss is 100 minus this. Row 18.
Re-acquisition time	Time from losing lock to regaining it; count, mean and maximum. A loss not regained by the end of the run counts in the maximum with its length so far. Row 19.
Slew saturation	Percentage of frames in which the commanded rate exceeded the gimbal limit.
Handoff ready	Locked with the estimate within 10 px (row 17) of the window centre, held 1 s: the point where fine pointing could take over; first time and share held.
Frames lost; lock on received frames	With simulated frame loss: the share of frames lost, and lock retention over the frames that arrived, beside the usual figure.
Segment	When the disturbances change during a run, each setting is a segment; tracking, vibration-removed and centroiding error, lock, tracked rate and FPS are also given per segment.
AI share	Percentage of frames in which the CNN provided the accepted measurement.

Appendix C. Command line reference

```

fsoc-tracker run --scenario configs/scenarios/<name>.yaml [--seed N | -1] [--duration S] [--out DIR]
fsoc-tracker video path/to/file.mp4 [--scenario cfg.yaml] [--truth truth.csv] [--out DIR]
fsoc-tracker batch --scenario a.yaml [b.yaml ...] --seeds 0-49 [--duration S] [--out DIR]
fsoc-tracker verify results/<run folder>
fsoc-tracker gui

```

Each run writes, into a folder named `FS0C_<sim|video>_<name>_seed-<N>_<date-time>` and with that label prefixed to every file, `frames.csv` (one row per frame, about 45 columns), `summary.json` (all metrics with definitions and pass or fail), `report.pdf` and `scenario.yaml` (the exact parameters, so the run can be repeated). `batch` adds `envelope.md` and `envelope.json`.

Appendix D. Problem statement parameter table, row by row

Row	Parameter	Suggested value	Implementation
1	Screen size (min.)	2000 x 2000, user-defined	<code>ScreenConfig.width/height</code> , default 2000 x 2000
2	Camera type	Monochrome FPA, colour optional	monochrome default; colour renders three channels, tracker uses luminance
3	Camera resolution	640 x 480, user-defined	<code>CameraConfig.width/height</code>
4	Camera FOV	user-defined, default 4 x 3 deg	<code>CameraConfig.fov_*</code> ; IFOV derived
5	Camera update rate	30 Hz min	<code>CameraConfig.update_rate_hz</code>
6	Initial camera position	centre of the screen	gimbal pose 0 at the screen centre
7	Target type	beacon spot	rendered spot with PSF
8	Number of targets	1 mandatory, multiple optional	<code>RunConfig.targets</code> , named; <code>designated</code> picks the one followed and scored; <code>designation</code> appearance, start or cue
9	Target shape	user-defined, default square	square, circle, gaussian, cross, ring, diamond, custom mask
10	Target size	5-20 x 5-20 px, default 10 x 10	<code>TargetConfig.size_px</code> (width) and <code>height_px</code>
11	Initial target location	user-defined, default random	random, centre, or "x,y" (panel or file)
12	Motion	line, circular, figure of 8, random; optional spiral, sinusoidal, user-defined	all six, a user-defined waypoint path, and static
13, 14	Max pan and tilt speed	5 to 10 deg/s, default 5	enforced in the gimbal model
15	Update interval	≥ 20 Hz	commands every frame at 30 Hz

Row	Parameter	Suggested value	Implementation
16	Acquisition time	≤ 2 s	measured, pass or fail printed
17	Tracking error	≤ 10 px	measured, pass or fail printed
18	Target loss	$< 5\%$	measured, pass or fail printed
19	Re-acquisition time	≤ 1 s	measured, pass or fail printed
20	Processing speed	≥ 20 FPS	measured, pass or fail printed
21	Image noise	salt and pepper about 10%, Gaussian, Poisson; one or more	three independent switches; salt and pepper shown in percent
22	Max standard deviation of noise	20, user-defined	<code>gaussian_sigma</code>
23	Max camera jitter	± 20 px per frame, user-defined	<code>jitter_px</code> , applied to the picture
24	Atmospheric disturbance	clear, haze, fog, rain, low light; user-defined contrast and brightness reduction	five presets plus editable contrast, brightness, blur, turbulence
25	Platform motion	± 20 px per frame max; linear mandatory; others optional	five bounded sway patterns with the configured peak speed (figure 8 scaled so its peak equals the setting)